

TALLER ANALÍTICO 1: FUNCIÓN ESCALÓN (15 MIN)

Dada la siguiente sub-matriz I de 3x3 (intensidades de gris):

80	120	140
90	200	210
50	130	250

1. Aplique analíticamente una umbralización binaria con $T = 135$. Escriba la matriz resultante (solo debe contener 0 y 255).
2. Si el objetivo era aislar todos los valores mayores a 100, ¿cuál fue el error de elegir $T=135$? Explique qué le ocurriría visualmente al objeto en la imagen segmentada.

1. Si el valor es mayor o igual a 135, se convierte en 255 (blanco). Si es menor a 135, se convierte en 0 (negro)

Matriz resultante:

0	0	225
0	255	255
0	0	255

2. Básicamente, el error fue poner el umbral demasiado alto (135 en lugar de 100). Al hacer eso, valores que sí cumplían con ser mayores a 100, como el 120 y el 130, terminaron tratándose como fondo y se volvieron 0. Visualmente, lo que pasa es que el objeto se ve incompleto o recortado. Como el umbral estaba muy arriba, se comió partes reales del objeto y dejó huecos, haciendo que la silueta pierda su forma original.

TALLER DE LABORATORIO FINAL: LIMPIANDO LA VISIÓN (45 MIN)

Desafío Práctico: Tratamiento de Ruido

1. Abran su IDE y carguen una imagen en escala de grises que tenga poco contraste (ej. un texto fotografiado con mala iluminación o una célula en microscopio).
2. Utilicen `cv2.threshold` con un valor estático que introduzca un poco de "ruido" intencional (puntos blancos no deseados en el fondo). Visualicen el resultado con `cv2.imshow`.
3. Construyan un Elemento Estructurante de 3x3.
4. Programen una operación morfológica de **Apertura (Erosión + Dilatación)** para limpiar el fondo sin encoger permanentemente el objeto principal.
5. Implementen una operación de **Cierre (Dilatación + Erosión)** en una variable diferente.
6. Muestren las 3 ventanas (Original binarizada, Apertura, Cierre) y concluyan: ¿Cuál operación morfológica fue más efectiva para su imagen específica y por qué?

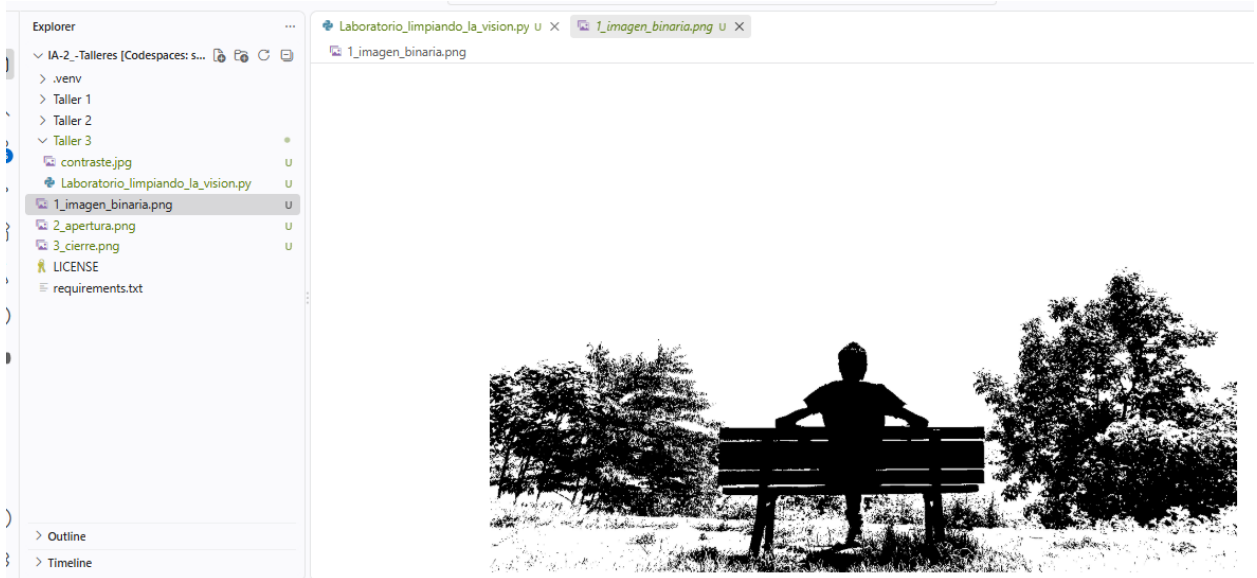
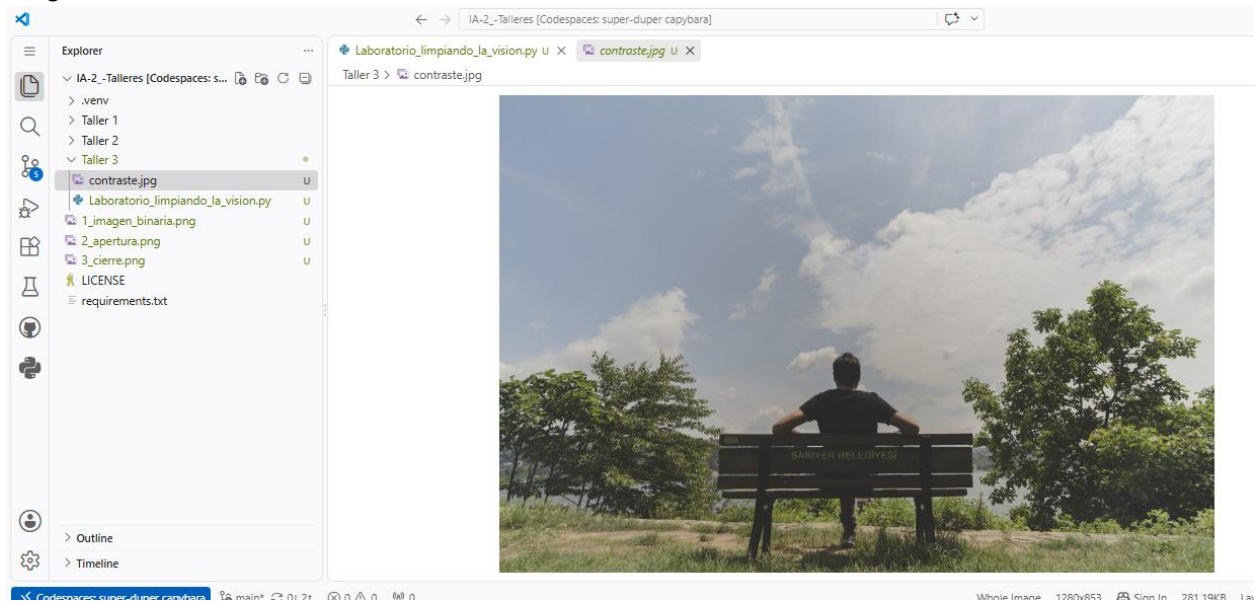
```

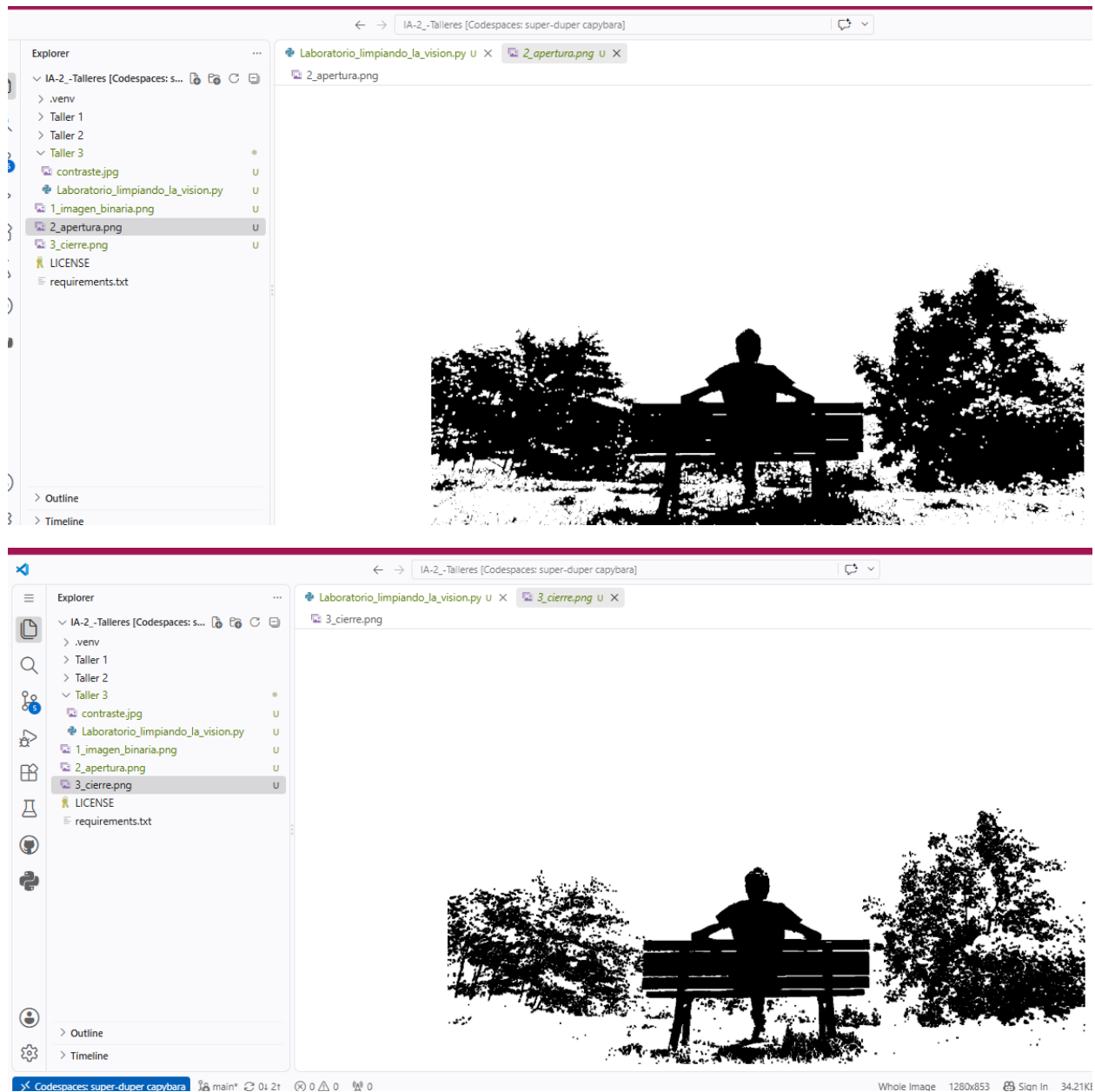
...
[Codespaces: super-duper capyb...
pg
p_limpiando_la_vision.py
inaria.png
ing
s.txt

Laboratorio_limpiando_la_vision.py U X
Taller 3 > Laboratorio_limpiando_la_vision.py > ...
1  import cv2
2  import numpy as np
3
4  imagen = cv2.imread('Taller 3/contraste.jpg', cv2.IMREAD_GRAYSCALE)
5  if imagen is None:
6      print("Error: no se pudo cargar la imagen.")
7  else:
8      print("Imagen cargada correctamente. Forma:", imagen.shape)
9
10 _, imagen_binaria = cv2.threshold(imagen, 90, 255, cv2.THRESH_BINARY)
11
12 cv2.imwrite('1_imagen_binaria.png', imagen_binaria)
13
14 kernel = np.ones((3, 3), np.uint8)
15
16 apertura = cv2.morphologyEx(imagen_binaria, cv2.MORPH_OPEN, kernel)
17 cv2.imwrite('2_apertura.png', apertura)
18
19 cierre = cv2.morphologyEx(imagen_binaria, cv2.MORPH_CLOSE, kernel)
20 cv2.imwrite('3_cierre.png', cierre)
21
22 print("Proceso completado. Revisa los 3 archivos generados:")
23 print("1_imagen_binaria.png -> Original binarizada con ruido")
24 print("2_apertura.png -> Resultado de Apertura")
25 print("3_cierre.png -> Resultado de Cierre")

```

Original:





Al final, probé las tres opciones y la Apertura (que es erosión seguida de dilatación) fue la que mejor funcionaba para quitarle el ruido a la foto.

La imagen original tenía un montón de puntitos blancos molestos en la zona de las plantas y los árboles de los lados. Lo bueno de la apertura es que borró esos puntos sueltos con la erosión y luego la dilatación acomodó el tamaño de los árboles y de la persona sin deformarlos, dejando todo mucho más limpio.

Por el contrario, el Cierre (dilatación y luego erosión) casi ni se notó; la imagen siguió igual de granulada. Tiene sentido porque el cierre sirve más para rellenar huecos por dentro, y el problema aquí eran puntos blancos volando por todas partes, no agujeros.

En resumen, como el ruido eran puras manchitas dispersas en las partes oscuras, la apertura terminó siendo la técnica perfecta para limpiar la imagen sin arruinar la forma de los elementos principales.